ChatGPT

Validation-First AI Strategy Pipeline for an AI-Generated North American QSR Top 30 “Core Data” Workbook

Executive summary

This project starts from a constraint made explicit in the Chinese chat screenshots: the “North American QSR Top 30 core data + AI strategy” file was produced by an AI system and **is not authoritative ground truth**. The ask is therefore not “analyze the numbers,” but to produce a **fully reproducible, programmatic credibility and validation system** that (1) detects hallucinations/fabrication signals, (2) reconciles metrics with verifiable reference sources, and (3) creates a clean, provenance-tagged “Gold” dataset that can safely underpin competitive analysis and downstream model building.

We began discovery using the enabled Hugging Face connector (api_tool) to identify high-leverage building blocks (time-series foundation forecasting models, RAG/fine-tuning tooling, and key papers). We then substantiated those findings using huggingface.com pages (papers, model cards, and docs), and used kaggle.com dataset cards to assemble development/benchmark datasets—especially those explicitly labeled synthetic, which are ideal for syntheticness detection evaluation. For example, the Chronos framework and model family provide a fast path to strong baseline forecasting with probabilistic outputs. [1]

The uploaded workbook itself contains three sheets (“QSR Top30 核心数据”, “AI策略与落地效果”, “数据说明与来源”) with chain-level metrics (units, system sales, AUV, estimated FTE, estimated store margin, supply chain model, ownership) plus a qualitative AI strategy matrix and a sources/notes sheet. A quick internal invariant check (implied AUV vs listed AUV) already shows large inconsistencies for at least some brands, indicating mixed definitions (e.g., U.S. vs global, systemwide vs subset, unit conversions, rounding). That is exactly the kind of issue this pipeline is designed to surface early—before any conclusions are drawn.

The plan below is “implementation-ready”: it defines assumptions and missing items as explicitly unspecified, details the data provenance model and sensitivity controls, specifies validation tests (schema, invariants, Benford, rounding/heaping, drift, MMD, and a supervised “syntheticness” classifier), includes reproducible Python code snippets and unit tests (pandas/Pandera/Great Expectations + PyTorch/Transformers + PEFT/LoRA + TRL), and lays out CI/data pipelines (DVC + GitHub Actions + MLflow). It also includes security/privacy alignment (NIST, PCI, CCPA, OWASP), deliverables, Gantt timeline, resource estimates, risks, and a handoff repo layout.

Key external anchors for reconciliation and governance include: - QSR Magazine’s QSR 50 report (U.S. systemwide sales, AUV, units, and collection notes). [2]
- Technomic’s Top 500 positioning/methodology and press releases for aggregate benchmarks (full dataset is paid). [3]
- SEC EDGAR official APIs for filings/XBRL-based facts and submission histories (for public companies). [4]

- FTC Franchise Rule constraints on financial performance representations if materials could be used in franchise/investor contexts. [5]

Key information needs

To deliver high-confidence outputs, we need the following 3–6 items clarified or obtained (currently unspecified unless otherwise noted):

- The **intended definitions** for each numeric field (e.g., "system sales" scope: U.S. only vs global; "units": U.S. only vs total; what fiscal year boundary). QSR 50 is explicitly U.S. systemwide sales; mismatches must be detected and normalized. [2]
- The **intended use** of outputs (internal strategy memo vs investor deck vs franchise materials). This affects provenance standards, legal disclaimers, and which numbers can be shown externally (FTC Item 19 constraints matter if you present earnings/sales claims). [5]
- Reference-source access level: whether you have a license to **Technomic Top 500 chain-level tables** (paid product) or will rely on public press releases + QSR 50 + filings. [6]
- Data for model workstreams (forecasting, labor optimization, RAG copilot): whether you will later ingest **store/SKU/day** operational data (POS, labor scheduling, inventory), and whether any of it contains PII (sensitivity classification).
- Compute budget and deployment constraints (GPU availability; cloud vs on-prem) are explicitly unspecified; the plan proposes tiered options without assuming one.

Context from artifacts and explicit assumptions

What the screenshots imply operationally

The Chinese chat screenshots establish intent and constraints: - The workbook is "AI-made not real data," and is being used as a "thought process / framework" rather than as truth. - The next value-add step is to write code that can **verify whether the data has hallucinations** (inconsistencies, fabrications, implausible patterns) before any strategic conclusions are drawn. - The deliverable expectation is a research-style analysis + reproducible verification approach that can be repeated for future datasets.

This changes the definition of "analysis": the primary output is a **credibility pipeline + evidence trail**, not a one-off narrative.

Findings from inspecting the uploaded Excel

The workbook structure is clean enough for systematic ETL: - **Sheet 1 ("QSR Top30 核心数据")**: 30 rows, 10 columns: rank, brand, category, U.S. store count (2024), system revenue ($B, 2024), AUV ($K), estimated daily-store FTE, estimated store margin, supply chain model, ownership model. - **Sheet 2 ("AI策略与落地效果")**: 31 rows, 6 columns: brand, AI strategy direction, key initiatives, deployment scale, outcomes, current status (2026Q1). - **Sheet 3 ("数据说明与来源")**: definitions for each field plus "key findings" bullets and a stated "data collection date" (2026-04-03).

A quick invariant check on Sheet 1 (AUV vs implied AUV = revenue/units) shows large discrepancies for some rows. This is not a "gotcha"; it is expected when AI-generated tables mix scopes or units. It proves the need for automated validation, tolerance rules, and source reconciliation.

GitHub repo inspection constraint

The request asked to begin by inspecting `https://github.com/charlie2233/RestaurantAnalysis` using a GitHub connector. In this environment, only the Hugging Face connector is enabled, and direct web fetching of that repo returned an error ("UnexpectedStatusCode"). Therefore, the plan below includes a **repo-agnostic handoff layout** and an explicit "integration slot" for the repo once it is provided as a zip/export or a GitHub connector is enabled.

Explicit assumptions

These are the only assumptions the plan makes; everything else is treated as unspecified.

- The uploaded workbook is a **hypothesis artifact**: it may contain incorrect numbers, mixed units, missing definitions, or invented estimates (especially for margin and FTE).
- Statistical fabrication tests (Benford, heaping, drift, MMD, classifier tests) are used as **signals**, not as proof. They must be interpreted with sample-size and data-generation caveats.
- Chain-level public data is often reported with rounding and non-uniform definitions; reconciliation tolerances are required, and provenance must be tracked per field.
- Legal/privacy controls must be designed now because later phases likely ingest operational data where PCI/CCPA/OWASP/NIST considerations become real constraints. [7]

Data needs, provenance, and sensitivity model

Data domains and required reference sources

The workbook mixes at least three concept types; each needs different validation methods:

- **Chain performance metrics** (units, U.S. systemwide sales, AUV): can be reconciled to structured public sources.
- QSR 50 provides a standardized table of U.S. sales (millions), AUV (thousands), and unit breakdowns plus methodology notes. [2]
- Technomic provides the leading Top 500 tracker (paid), with public press releases that can serve as sanity checks for aggregate claims. [3]

- SEC EDGAR APIs can provide filings histories and XBRL facts for public companies, enabling "primary-source anchoring" for some performance measures. [4]

- **Operational estimates** (store-level margin, FTE): often not publicly disclosed at comparable granularity and frequently require estimation. These must be tagged as "derived" unless you have internal data or a defensible source.

- **AI strategy narratives** (initiatives, rollout status, "impact" claims): these require traceable citations to press releases, earnings calls, or credible reporting. Example anchors:

- McDonald's ended its IBM automated order taker test in 2024 (external evidence for the workbook's "AOT terminated" narrative). [8]

- Starbucks publicly introduced Green Dot Assist and described it as a real-time barista assistant on in-store iPads (external evidence for "internal copilot" strategy). [9]
- Chipotle's IR press release describes Autocado and the Augmented Makeline and provides quantitative and pilot details (external evidence for "cobotics" strategy). [10]

Provenance schema and confidence scoring

Every field in the eventual "Gold" dataset must carry:

- `source_type`: {primary_filing, official_company_release, industry_report, reputable_media, derived_estimate, AI_generated}
- `source_locator`: URL, accession number (EDGAR), or document ID
- `as_of_date`: the exact date the figure reflects
- `scope`: {US, NorthAmerica, global}, `definition_version`
- `confidence`: numeric 0–1 driven by rules (below)
- `validation_status`: {pass, warn, fail} with error diagnostics

A minimal scoring rubric:

Provenance grade	Source type	Confidence guideline	Example
A	Primary/official	0.9–1.0	SEC filing via EDGAR API [11]
B	Industry report	0.75–0.9	QSR 50 values; QSR notes "some estimated by QSR" [2]
C	Reputable journalism	0.6–0.8	CNBC reporting on program changes [8]
D	Derived estimate	0.3–0.6	Computed margins/FTE without primary data
E	AI-generated/ unknown	0.0–0.3	Workbook numbers without sources (current state)

Sensitivity classification

Even if the current workbook is synthetic, the system should treat future datasets as potentially sensitive:

- **Public**: published tables (QSR 50 headline numbers), public press releases.
- **Internal**: your reconciled dataset + credibility scores; strategy notes; vendor comparisons.
- **Confidential/Restricted**: store-level sales, labor hours, cost data, supplier terms; customer data; card payment flows (PCI scope).

PCI matters if cardholder data ever touches your ecosystem; requirement 4 explicitly calls for strong cryptography for transmission over open/public networks. [12]
CCPA regulations provide operational guidance on compliance and consumer rights processes in California contexts. [13]

OWASP API Top 10 highlights common API failure modes (e.g., broken object-level authorization) that are directly relevant if you serve dashboards or inference APIs. [14]
NIST SP 800-53 provides a comprehensive control catalog baseline for access control, auditing, incident response, and other system protections. [15]

Kaggle and Hugging Face datasets for development and benchmarking

To build and test the pipeline (especially syntheticness detection) without relying on proprietary QSR data, use these:

- Kaggle's "Store Item Demand Forecasting Dataset" is explicitly labeled **synthetic** and describes a generation process (trend/seasonality/promo/noise). It is ideal for developing "syntheticness" detectors and drift tests. [16]
- Kaggle's "Fast Food Restaurants Across America" provides restaurant location attributes and can be used for density/geo feature pipelines. [17]
- Kaggle's "Recruit Restaurant Visitor Forecasting" dataset (legacy benchmark) can support early forecasting prototypes (note license/description limitations). [18]

On Hugging Face: - Chronos paper and model cards provide time-series foundation forecasting models (including Chronos-2). [1]
- PEFT docs define parameter-efficient fine-tuning (including LoRA support). [19]
- TRL SFTTrainer docs provide supervised fine-tuning scaffolding and configuration options. [20]

Validation and fabricated-data detection system

System objective and operating principle

The validation system must produce two things:

1) **Deterministic failures** (schema and invariants): "This cannot be correct under its own definitions." 2) **Probabilistic risk signals** (Benford/heaping/drift/MMD/C2ST): "This pattern is unusual compared to reference data."

Only the first category should automatically block a dataset from being published to Gold. The second category should raise warnings and require human review or reconciliation.

Programmatic validation flow

```
flowchart TD
  A[Ingest: workbook + reference sources] --> B[Bronze storage: raw snapshots]
  B --> C[Normalize + standardize units/definitions]
  C --> D[Schema validation: Pandera + GE]
  D -->|fail| Q[Quarantine + issue ticket + diagnostics]
  D -->|pass| E[Invariant checks: arithmetic/logic constraints]
  E -->|fail| Q
  E -->|pass| F[Statistical signals: Benford + heaping + drift + MMD + C2ST]
```

```
F --> G[Reconciliation: join to QSR50/Technomic/EDGAR]
G --> H[Credibility scorecards per field + per brand]
H --> I[Gold dataset publish + versioning]
I --> J[Downstream: analytics + forecasting + RAG]
```

Validation method comparison table

Test	Detects	Data requirements	Output	Strengths	Limitations
Schema checks (Pandera/GE)	Missing columns, type errors, range violations	Any dataset	Pass/fail + errors	Deterministic, CI-friendly	Doesn't detect plausible-but-wrong
Invariants (arithmetic + consistency)	Internal contradictions (e.g., AUV vs sales/units)	Any dataset with related fields	Pass/fail + row diagnostics	High precision for hallucinations	Requires correct definitions
Benford + conformity tests	Digit distribution anomalies	Best with large N and multi-order-of-magnitude values	p-values, MAD, χ^2	Fast "screening" method; widely used in audits [21]	Can be invalid if data is bounded/ rounded/ small N
Rounding/ heaping tests	Excess "nice numbers" (0/5/00 endings)	Numeric cols	End-digit distribution, heaping score	Simple, interpretable	Heaping can be legitimate reporting practice
Drift tests (KS/χ^2/PSI)	Distribution shift vs reference	Reference distribution	Drift flags + distances	Standard monitoring pattern	Needs good reference sample
MMD (kernel two-sample)	Multivariate distribution difference	Two samples	Test statistic + p-value	Strong nonparametric test [22]	Sensitive to kernel/ bandwidth; compute cost

Test	Detects	Data requirements	Output	Strengths	Limitations
Supervised syntheticness classifier (C2ST)	Learnable difference between “real” and “synthetic”	Labeled real/synthetic; or two datasets for two-sample classification	AUC/accuracy; feature importance	Scalable; interpretable where differences arise (C2ST framing) [23]	Needs dataset zoo; risk of shortcut learning

Reproducible code: loading and normalizing the workbook

```
import pandas as pd

XLSX_PATH = "北美QSR_Top30_核心数据_AI策略.xlsx"

def load_workbook(path: str) -> dict[str, pd.DataFrame]:
    core = pd.read_excel(path, sheet_name="QSR Top30 核心数据")
    ai = pd.read_excel(path, sheet_name="AI策略与落地效果")
    notes = pd.read_excel(path, sheet_name="数据说明与来源")
    return {"core": core, "ai": ai, "notes": notes}

def normalize_columns(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df.columns = [c.replace("\n", "").strip() for c in df.columns]
    return df
```

Schema validation with Pandera (CI-friendly)

Pandera’s DataFrame schema model and checks support declarative validation and integrate naturally into unit tests. [24]

```
import pandera.pandas as pa
from pandera import Column, Check

CoreSchema = pa.DataFrameSchema(
    {
        "排名": Column(int, Check.in_range(1, 60), nullable=False),
        "品牌": Column(str, Check.str_length(1, 200), nullable=False),
        "品类": Column(str, nullable=True),
        "美国门店数(2024)": Column(int, Check.ge(0), nullable=False),
        "全系统营收($B, 2024)": Column(float, Check.ge(0), nullable=False),
        "店均AUV($K)": Column(float, Check.gt(0), nullable=False),
        "店均日等效FTE(估算)": Column(float, Check.gt(0), nullable=True),
        "门店利润率(估算)": Column(str, nullable=True),
```

```
# parse later into numeric bounds
        "央厨/供应链模式": Column(str, nullable=True),
        "所有制模式": Column(str, nullable=True),
    },
    strict=False,
    coerce=True,
)

def validate_core_schema(df_core: pd.DataFrame) -> pd.DataFrame:
    df_core = normalize_columns(df_core)
    return CoreSchema.validate(df_core)
```

Great Expectations checkpoint setup (production validation)

Great Expectations Checkpoints are a primary "production validation" abstraction and can bundle suite + actions. [25]

```
# Pseudocode / skeleton: actual GE project bootstrap requires
`great_expectations init`
import great_expectations as gx

def run_checkpoint(data_path: str, suite_name: str, checkpoint_name: str):
    context = gx.get_context()
    checkpoint = context.get_checkpoint(checkpoint_name)
    return checkpoint.run()
```

Invariant checks (hard contradiction detection)

These checks should be parameterized by "definition profile" (e.g., US-only vs global). The QSR 50 defines U.S. systemwide sales explicitly. [2]

```
import numpy as np

def add_derived_fields(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    # Convert sales $B to $K per store: $B * 1e9 / stores / 1e3
    df["implied_auv_k"] = (df["全系统营收($B, 2024)"] * 1e9) / df["美国门店数
(2024)"] / 1e3
    df["auv_rel_err"] = (df["implied_auv_k"] - df["店均AUV($K)"]) / df["店均
AUV($K)"]
    return df

def flag_invariant_violations(df: pd.DataFrame, rel_tol: float = 0.05) ->
pd.DataFrame:
    df = df.copy()
```

```
    df["auv_invariant_ok"] = df["auv_rel_err"].abs() <= rel_tol
    return df
```

Benford analysis (first digit / first-two digits) with conformity tests

A MethodsX open-access implementation describes using Benford first/first-two digits and multiple conformity tests (χ^2, Z-stat, MAD) as a fraud/anomaly screen. [21]

```
import numpy as np
from scipy.stats import chisquare

BENFORD_P = np.array([np.log10(1 + 1/d) for d in range(1, 10)])

def first_digit(arr):
    x = np.asarray(arr, dtype=float)
    x = x[np.isfinite(x) & (x != 0)]
    x = np.abs(x)
    d = np.floor(x / (10 ** np.floor(np.log10(x)))).astype(int)
    return d[(d >= 1) & (d <= 9)]

def benford_chi2(arr):
    d = first_digit(arr)
    counts = np.array([(d == k).sum() for k in range(1, 10)])
    n = counts.sum()
    if n < 50:
        return {"n": int(n), "chi2": None, "p": None, "counts": counts}
    expected = BENFORD_P * n
    chi2, p = chisquare(counts, f_exp=expected)
    return {"n": int(n), "chi2": float(chi2), "p": float(p), "counts": counts}
```

Unit tests (pytest-style) to ensure the detector is not trivially broken:

```
def test_benford_on_loguniform():
    rng = np.random.default_rng(0)
    x = 10 ** rng.uniform(0, 6, size=20000)
    res = benford_chi2(x)
    assert res["n"] > 1000
    assert res["p"] is not None

def test_benford_on_constant_should_skip():
    x = np.ones(10) * 1000
    res = benford_chi2(x)
    assert res["chi2"] is None
```

Rounding/heaping detection

Heaping is most visible via end-digit distributions and spikes at “nice” thresholds. Even if not fraudulent, it signals low-precision reporting or synthetic rounding.

```
import pandas as pd

def trailing_digit(x, base=10):
    x = pd.Series(x).dropna().astype(float).abs()
    # scale to integer if needed before mod; here assume already integer-like
    return (x.round(0).astype(int) % base)

def heaping_score(arr, digits=(0, 5), base=10):
    d = trailing_digit(arr, base=base)
    if len(d) == 0:
        return None
    return float(d.isin(list(digits)).mean())  # fraction ending in 0 or 5
```

Drift tests and monitoring hooks

For the chain-level table, drift tests are mainly meaningful when you have a larger reference set (e.g., all QSR 50 rows, multi-year). The system should support: - KS test for continuous - χ^2 test for categorical - PSI for monitoring dashboards (optional)

MMD (kernel two-sample test) with permutation p-value

MMD is a standard kernel two-sample test with strong theoretical grounding. [22]

```
import numpy as np

def rbf_kernel(X, Y, gamma):
    XX = np.sum(X**2, axis=1)[:, None]
    YY = np.sum(Y**2, axis=1)[None, :]
    distances = XX + YY - 2 * (X @ Y.T)
    return np.exp(-gamma * distances)

def mmd2_unbiased(X, Y, gamma=1.0):
    Kxx = rbf_kernel(X, X, gamma)
    Kyy = rbf_kernel(Y, Y, gamma)
    Kxy = rbf_kernel(X, Y, gamma)
    n = X.shape[0]
    m = Y.shape[0]
    np.fill_diagonal(Kxx, 0.0)
    np.fill_diagonal(Kyy, 0.0)
    return (Kxx.sum() / (n * (n - 1))
```

```
            + Kyy.sum() / (m * (m - 1))
            - 2 * Kxy.mean())

def mmd_permutation_test(X, Y, gamma=1.0, n_perm=1000, seed=0):
    rng = np.random.default_rng(seed)
    Z = np.vstack([X, Y])
    n = X.shape[0]
    stat = mmd2_unbiased(X, Y, gamma)
    count = 0
    for _ in range(n_perm):
        idx = rng.permutation(Z.shape[0])
        Xp = Z[idx[:n]]
        Yp = Z[idx[n:]]
        if mmd2_unbiased(Xp, Yp, gamma) >= stat:
            count += 1
    pval = (count + 1) / (n_perm + 1)
    return stat, pval
```

Supervised syntheticness classifier

There are two complementary implementations:

- **C2ST framing**: train a classifier to distinguish dataset A vs dataset B; if accuracy is near chance, distributions look similar; if high, they differ. This is the core idea in "Revisiting classifier two-sample tests." [23]
- **Real-vs-synthetic classifier**: build a training zoo with multiple synthetic generators (CTGAN, Gaussian copula, simple rule-based, LLM-based) and multiple real sources; predict `is_synthetic`.

CTGAN tooling in SDV provides a practical synthetic generator for tabular data. [26]

PyTorch classifier skeleton (feature-based):

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

class MLP(nn.Module):
    def __init__(self, d_in: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, 64),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
```

```
        )
    def forward(self, x):
        return self.net(x).squeeze(-1)

def train_binary_classifier(X_train, y_train, X_val, y_val, epochs=20, lr=1e-3):
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = MLP(X_train.shape[1]).to(device)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.BCEWithLogitsLoss()

    train_loader = DataLoader(
        TensorDataset(torch.tensor(X_train, dtype=torch.float32),
                      torch.tensor(y_train, dtype=torch.float32)),
        batch_size=64, shuffle=True
    )

    for _ in range(epochs):
        model.train()
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            logits = model(xb)
            loss = loss_fn(logits, yb)
            loss.backward()
            opt.step()

    model.eval()
    with torch.no_grad():
        logits = model(torch.tensor(X_val,
dtype=torch.float32).to(device)).cpu().numpy()
    return model, logits
```

Evaluation metrics for this classifier: - ROC-AUC, PR-AUC - Calibration (Brier score / reliability curves) - Cross-generator generalization: train on generator types A/B/C, test on unseen synthetic type D (prevents "shortcut learning").

Model training and fine-tuning plan

This section is downstream of validation: models must consume **Gold** tables only, never raw AI-generated fields without provenance.

Forecasting: demand, staffing, inventory

Chronos provides a strong baseline for probabilistic forecasting and supports zero-shot transfer across datasets by tokenizing time series values and training transformer architectures. [1]

Recommended approach: - Establish classical baselines first (seasonal naive, ETS/ARIMA, gradient boosting with lag features). - Add Chronos as a foundation model baseline. - Only fine-tune deep models after you have stable data definitions and evaluation harness.

Forecast evaluation: - Point metrics: MAE, RMSE, wMAPE (avoid plain MAPE with small denominators) - Probabilistic metrics: pinball loss for quantiles; coverage of prediction intervals.

Ops copilot via Retrieval-Augmented Generation

RAG combines parametric generation with non-parametric retrieval and improves factual grounding for knowledge-intensive outputs. [27]

Design: - Ingest internal SOPs, equipment manuals, recipe cards, policy docs into a vector store. - Enforce citation-in-answer behavior (responses must quote or reference retrieved chunks). - Add "refusal behaviors" when retrieval confidence is low or conflicts exist.

Fine-tuning strategy: PEFT/LoRA + TRL SFTTrainer

Use PEFT to reduce training cost by fine-tuning adapters rather than full weights. [19]
LoRA is the canonical low-rank adapter method and is well documented in the original paper. [28]
TRL's SFTTrainer provides supervised fine-tuning scaffolding and supports packing, assistant-only loss, and integration patterns. [20]

Minimal SFT template:

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig

base_model = "Qwen/Qwen3-0.6B"   # placeholder; choose based on policy+compute
tokenizer = AutoTokenizer.from_pretrained(base_model)
model = AutoModelForCausalLM.from_pretrained(base_model, device_map="auto")

lora_cfg = LoraConfig(
    r=8, lora_alpha=32, lora_dropout=0.05,
    bias="none", task_type="CAUSAL_LM",
)

train_ds = load_dataset("trl-lib/Capybara", split="train")  # placeholder
trainer = SFTTrainer(
    model=model,
    train_dataset=train_ds,
    args=SFTConfig(
        output_dir="runs/qsr_ops_sft",
        learning_rate=1e-4,
```

```
        per_device_train_batch_size=2,
        gradient_accumulation_steps=8,
        num_train_epochs=1,
        packing=True,
        assistant_only_loss=True,
    ),
    peft_config=lora_cfg,
    tokenizer=tokenizer,
)
trainer.train()
trainer.save_model()
```

Industry-informed risk gating for customer-facing AI

Use external case evidence to set rollout guardrails: - McDonald's ended its IBM AOT test after issues and shifted to exploring other solutions; this supports a "do not lead with drive-thru voice automation" stance unless accuracy and unit economics are proven. [8]
- Starbucks positions AI as partner enablement; Green Dot Assist is a barista tool piloted in limited stores and framed explicitly as strengthening human connection. [9]
- Chipotle frames cobotics as a staged pilot with operational feedback loops and explicit locations/metrics—an example of "stage-gate" governance for automation. [10]

Engineering plan, CI/data pipeline, security, deliverables, timeline, and risks

DVC + pipeline reproducibility

DVC pipelines are defined in `dvc.yaml` as stages forming a DAG; DVC tracks dependencies/outputs and supports reproducible reruns. [29]

Example `dvc.yaml` skeleton:

```
stages:
  ingest:
    cmd: python -m src.ingest.workbook --in data/raw/北美QSR_Top30_核心数据_AI策略.xlsx --out data/bronze/core.parquet
    deps:
      - src/ingest/workbook.py
      - data/raw/北美QSR_Top30_核心数据_AI策略.xlsx
    outs:
      - data/bronze/core.parquet

  validate:
    cmd: python -m src.validate.run_all --in data/bronze/core.parquet --out reports/validation.json
```

```
    deps:
      - src/validate/run_all.py
      - src/validate/pandera_schemas.py
    outs:
      - reports/validation.json

  reconcile_qsr50:
    cmd: python -m src.reconcile.qsr50 --out data/silver/qsr50.parquet
    deps:
      - src/reconcile/qsr50.py
    outs:
      - data/silver/qsr50.parquet

  score:
    cmd: python -m src.score.credibility --core data/bronze/core.parquet --ref
data/silver/qsr50.parquet --out data/gold/qsr_top30_gold.parquet
    deps:
      - src/score/credibility.py
      - reports/validation.json
    outs:
      - data/gold/qsr_top30_gold.parquet
```

GitHub Actions CI

GitHub provides a standard guide for building/testing Python in Actions. [30]

Example workflow excerpt:

```
name: ci

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest -q
```

MLflow experiment tracking and registry

MLflow Model Registry offers centralized versioning, lineage, and governance for models and their runs. [31]

Example logging snippet:

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("rel_tol", 0.05)
    mlflow.log_metric("pct_invariant_fail", 0.12)
    mlflow.log_artifact("reports/validation.json")
    # mlflow.<flavor>.log_model(...) for models
```

Security and privacy controls mapping

- NIST SP 800-53 provides the control catalog baseline for access control, audit/logging, incident response, and more (useful as a control checklist). [15]
- OWASP API Top 10 (2023) explicitly calls out broken object-level authorization and related access control failures—critical if you serve dashboards or inference endpoints. [14]
- PCI guidance highlights the need for strong cryptography for transmitting cardholder data over open/public networks (scope if payments enter your system). [12]
- CCPA regulations provide operational guidance for consumer rights processes and compliance; design data deletion/export workflows early if customer data will be processed. [13]

Minimum security requirements for the MVP: - Data encryption in transit + at rest; secrets management; least-privilege IAM; audit logs for dataset access and model inference. - Clear separation between "public reference data" and "internal/confidential operational data." - API rate limiting and anti-automation controls for sensitive business flows (aligned with OWASP API6:2023 concept). [32]

Deliverables

Data credibility and validation deliverables - `data_dictionary.md` with explicit field definitions, units, geography, and tolerance rules. - Pandera schemas + unit tests for each table. - Great Expectations suites + checkpoints and rendered Data Docs. - "Credibility scorecard" report: - per brand, - per field, - provenance grade, - invariant pass/fail, - statistical risk signals (Benford/heaping/drift/MMD/C2ST), - reconciliation deltas vs QSR 50 / filings.

Reference ingestion deliverables - QSR 50 extractor (HTML table parsing) and reconciler to the core table. [2]
- Technomic integration stub: - public press-release sanity checks by default, plus an interface layer to load paid tables if licensed. [33]
- SEC EDGAR API ingestion module (company submissions and/or XBRL facts) with fair-access request throttling. [4]

Modeling deliverables - Forecasting baseline suite + Chronos benchmark notebook and evaluation harness. [1]
- RAG ops-copilot MVP (document ingestion + retrieval + grounded responses) with evaluation set and hallucination tests. [27]
- Optional SFT package: LoRA adapters + TRL SFTTrainer config + training/eval scripts (gated behind privacy review). [34]

Compliance deliverables - "External use readiness checklist" referencing FTC constraints on financial performance representations (reasonable basis + substantiation). [5]

Handoff repo layout

```
RestaurantAnalysis/
  README.md
  requirements.txt
  pyproject.toml
  dvc.yaml
  params.yaml

  data/
    raw/                      # immutable snapshots (Bronze)
    bronze/
    silver/
    gold/

  src/
    ingest/
      workbook.py
      qsr50.py
      sec_edgar.py
    normalize/
      units.py
      names.py                # brand normalization + entity resolution
    validate/
      pandera_schemas.py
      ge/                     # GE suites + checkpoint configs
      invariants.py
      digit_tests.py
      heaping.py
      drift.py
      mmd.py
      c2st.py
    reconcile/
      qsr50_join.py
      scoring.py
    models/
      forecasting/
```

```
      rag/
      sft/
    security/
      data_classification.md
      threat_model.md

  tests/
    test_schemas.py
    test_invariants.py
    test_benford.py
    test_mmd.py
    test_c2st.py

  reports/
    validation/
    reconciliation/
    scorecards/
```

Timeline (Gantt)

```
gantt
  title QSR Top30 Credibility Pipeline and AI Strategy MVP
  dateFormat  YYYY-MM-DD
  axisFormat  %b %d

  section Foundations
  Repo skeleton + CI scaffolding               :a1, 2026-04-07, 5d
  Data dictionary + definition profiles        :a2, after a1, 7d

  section Validation system
  Pandera schemas + unit tests                 :b1, 2026-04-14, 7d
  GE suites + checkpoints + Data Docs          :b2, after b1, 7d
  Invariants + contradiction diagnostics       :b3, after b1, 5d

  section Fabrication signals
  Benford + rounding/heaping module            :c1, 2026-04-24, 5d
  Drift tests + reference hooks                :c2, after c1, 5d
  MMD + C2ST classifier tests                  :c3, after c2, 7d

  section Provenance + reconciliation
  QSR50 extraction + reconciliation scoring     :d1, 2026-05-08, 10d
  SEC EDGAR ingestion module (optional)        :d2, after d1, 10d

  section Modeling MVP
  Forecasting baseline + Chronos benchmark     :e1, 2026-05-22, 10d
  RAG ops copilot prototype + eval harness     :e2, 2026-05-22, 10d
```

```
section Hardening + handoff
Security/privacy controls + documentation     :f1, 2026-06-02, 7d
Final scorecards + investor-safe export gate :f2, after f1, 5d
```

Resource estimates

Because compute and staffing are unspecified, the plan defines a minimum viable team and optional add-ons:

- Minimum:
- 1 data/analytics engineer (ETL, validation, reconciliation)
- 1 ML engineer (forecasting + RAG prototype)
- 0.25 security reviewer (threat model + access controls)
- Optional:
- 1 MLOps engineer (deployment, monitoring, scaling, infra)
- Domain analyst (finance/ops definitions, tolerance rules)

Compute tiers: - Low: CPU-only for validation + RAG embeddings; no fine-tuning. - Medium: 1 GPU for embeddings and light LoRA runs. - High: multi-GPU for faster training and larger context.

Risk register and mitigations

- **Risk: treating AI-generated metrics as factual in external materials**
- Mitigation: hard "Gold-only export gate" + per-field provenance + confidence; FTC "reasonable basis" and substantiation requirements inform gating if the content resembles financial performance representations. [5]
- **Risk: false positives from Benford/heaping on small N**
- Mitigation: label statistical tests as "signals," require corroboration with invariants + reconciliation; enforce minimum sample-size rules (skip or downweight when N is small). [21]
- **Risk: incomplete access to Technomic chain-level tables**
- Mitigation: design reconciliation adapters so public sources (QSR 50 + SEC + press releases) work by default; paid sources can be "plugged in" later. [6]
- **Risk: API security failures in dashboards/inference services**
- Mitigation: OWASP API Top 10-driven design review; enforce object-level authorization, least privilege, rate limits, and logging. [14]
- **Risk: future privacy violations once operational/customer data is ingested**

- Mitigation: early data classification, retention controls, deletion workflows, and CCPA request handling patterns. [13]

Immediate next steps

- Implement the “definition profile” layer (U.S. vs global; systemwide vs subset) and run invariants under each profile, flagging which profile yields the least contradictions.
- Build QSR 50 ingestion and reconciliation first, because it provides a structured table and explicit methodology notes suitable for automated joins and tolerance scoring. [2]
- Add statistical signal modules (Benford/heaping/drift/MMD/C2ST) behind a “review required” status by default; never auto-block solely on these signals.
- Enforce a repository rule: no numbers may reach “Gold exports” without `source_type`, `source_locator`, `as_of_date`, and `confidence` populated.

---

[1] https://huggingface.co/papers/2403.07815
https://huggingface.co/papers/2403.07815

[2] https://www.qsrmagazine.com/story/the-2025-qsr-50-fast-foods-leading-annual-reportqsr-50-2025/
https://www.qsrmagazine.com/story/the-2025-qsr-50-fast-foods-leading-annual-reportqsr-50-2025/

[3] https://www.technomic.com/report-top-500/
https://www.technomic.com/report-top-500/

[4] [11] https://www.sec.gov/edgar/sec-api-documentation
https://www.sec.gov/edgar/sec-api-documentation

[5] https://www.law.cornell.edu/cfr/text/16/436.5
https://www.law.cornell.edu/cfr/text/16/436.5

[6] https://store.technomic.com/products/top-500-chain-restaurant-report-3
https://store.technomic.com/products/top-500-chain-restaurant-report-3

[7] [15] https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final
https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

[8] https://www.cnbc.com/2024/06/17/mcdonalds-to-end-ibm-ai-drive-thru-test.html
https://www.cnbc.com/2024/06/17/mcdonalds-to-end-ibm-ai-drive-thru-test.html

[9] https://about.starbucks.com/press/2025/meet-green-dot-assist-starbucks-generative-ai-powered-coffeehouse-companion/
https://about.starbucks.com/press/2025/meet-green-dot-assist-starbucks-generative-ai-powered-coffeehouse-companion/

[10] https://ir.chipotle.com/2024-09-16-CHIPOTLE-DEBUTS-AUTOCADO-AND-THE-AUGMENTED-MAKELINE-BY-HYPHEN-IN-RESTAURANTS
https://ir.chipotle.com/2024-09-16-CHIPOTLE-DEBUTS-AUTOCADO-AND-THE-AUGMENTED-MAKELINE-BY-HYPHEN-IN-RESTAURANTS

[12] https://www.pcisecuritystandards.org/faq/articles/Frequently_Asked_Question/is-it-permissible-to-use-self-decrypting-files-for-encryption-to-send-cardholder-data/
https://www.pcisecuritystandards.org/faq/articles/Frequently_Asked_Question/is-it-permissible-to-use-self-decrypting-files-for-encryption-to-send-cardholder-data/

13 https://oag.ca.gov/privacy/ccpa/regs
https://oag.ca.gov/privacy/ccpa/regs

14 32 https://owasp.org/API-Security/editions/2023/en/0x11-t10/
https://owasp.org/API-Security/editions/2023/en/0x11-t10/

16 https://www.kaggle.com/datasets/dhrubangtalukdar/store-item-demand-forecasting-dataset
https://www.kaggle.com/datasets/dhrubangtalukdar/store-item-demand-forecasting-dataset

17 https://www.kaggle.com/datasets/datafiniti/fast-food-restaurants
https://www.kaggle.com/datasets/datafiniti/fast-food-restaurants

18 https://www.kaggle.com/datasets/dpascual/recruitrestaurantvisitorforecasting
https://www.kaggle.com/datasets/dpascual/recruitrestaurantvisitorforecasting

19 https://huggingface.co/docs/transformers/peft
https://huggingface.co/docs/transformers/peft

20 https://huggingface.co/docs/trl/en/sft_trainer
https://huggingface.co/docs/trl/en/sft_trainer

21 https://pmc.ncbi.nlm.nih.gov/articles/PMC8720889/
https://pmc.ncbi.nlm.nih.gov/articles/PMC8720889/

22 https://jmlr.org/beta/papers/v13/gretton12a.html
https://jmlr.org/beta/papers/v13/gretton12a.html

23 https://github.com/lopezpaz/classifier_tests
https://github.com/lopezpaz/classifier_tests

24 https://pandera.readthedocs.io/en/latest/dataframe_schemas.html
https://pandera.readthedocs.io/en/latest/dataframe_schemas.html

25 https://docs.greatexpectations.io/docs/0.18/reference/learn/terms/checkpoint
https://docs.greatexpectations.io/docs/0.18/reference/learn/terms/checkpoint

26 https://docs.sdv.dev/sdv/single-table-data/modeling/synthesizers/ctgansynthesizer
https://docs.sdv.dev/sdv/single-table-data/modeling/synthesizers/ctgansynthesizer

27 https://huggingface.co/papers/2005.11401
https://huggingface.co/papers/2005.11401

28 34 https://huggingface.co/papers/2106.09685
https://huggingface.co/papers/2106.09685

29 https://dvc.org/doc/user-guide/pipelines/defining-pipelines
https://dvc.org/doc/user-guide/pipelines/defining-pipelines

30 https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python
https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python

31 https://www.mlflow.org/docs/3.0.1/model-registry
https://www.mlflow.org/docs/3.0.1/model-registry

33 https://www.technomic.com/press-top-500-chain-restaurant-sales-growth-slows-to-3-in-2024-per-technomics-annual-report/
https://www.technomic.com/press-top-500-chain-restaurant-sales-growth-slows-to-3-in-2024-per-technomics-annual-report/